PaperPilot – GenAI Research Paper Assistant

Software Requirements Specification (SRS)

Team: Team-1

Team Members: Harish Kumar | Kalpana Bhardwaj | Prakriti | Vansh
Faculty Superviser: Mr. Saqib

Mentor: Mr. Zuber

University: Jagannath University

1. Introduction

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive, detailed, and technical description of the PaperPilot system. This document outlines the architectural blueprint, core functional requirements, non-functional constraints, and the underlying logic of the Retrieval-Augmented Generation (RAG) pipeline. This SRS serves as the definitive reference manual for core developers, system architects, software testers, and academic stakeholders to thoroughly understand the system's execution flow, interface design, and deployment prerequisites without requiring direct access to the source code.

1.2 Scope

PaperPilot is an advanced, **web-based, AI-powered research assistant** engineered to mitigate the prevalent issue of information overload in academic and professional research environments. By leveraging state-of-the-art Generative AI techniques, the system enables users to upload lengthy, complex research papers (PDFs) and extract highly accurate, context-bound insights. Unlike traditional chatbots that rely solely on pre-trained data (which often leads to hallucinations), PaperPilot strictly restricts its answers to the uploaded document's context.

The comprehensive scope of the system provides the following capabilities:

- **Secure User Authentication:** Session-state driven login and registration module to ensure workspace privacy.
- **Automated Document Ingestion:** Seamless uploading and parsing of extensive research papers (PDF format) utilizing automated text extraction algorithms.
- **AI-Generated Summarization:** Rapid generation of high-level abstracts by processing the initial contextual data of the uploaded documents.
- **Structured Notes Generation:** Intelligent extraction and formatting of critical information segregated into Introduction, Key Points, and Conclusion.
- **Context-Aware Chatbot (RAG-Based):** An interactive query interface backed by a local vector database, allowing users to ask highly specific questions and receive accurate, pinpointed answers derived exclusively from the uploaded PDF text.

1.3 Intended Audience

This document is specifically curated and intended for the following stakeholders:

- **Developers & System Architects:** To understand the implementation of the LangChain framework, ChromaDB vectorization, and LLM orchestration.
- **Project Evaluators / Academic Jury:** To assess the technical depth, practical application of the RAG pipeline, and overall project viability.
- **Researchers / Students:** To understand the capabilities and limitations of the system as end-users leveraging the AI functionalities.
- **System Testers / QA Engineers:** To design rigorous test cases for evaluating UI responsiveness, PDF parsing limits, and LLM inference accuracy.

1.4 Definitions & Acronyms

- **AI (Artificial Intelligence):** The simulation of human cognitive processes by algorithms to perform tasks requiring intelligent reasoning.
- **RAG (Retrieval-Augmented Generation):** An advanced AI framework that improves the accuracy and reliability of LLM-generated responses by continuously fetching facts from an external, user-provided knowledge base (e.g., the uploaded PDF).
- **LLM (Large Language Model):** A deep learning algorithm capable of recognizing, summarizing, translating, predicting, and generating text. PaperPilot utilizes the open-source Llama-3.3-70b model via the Groq API.
- **PDF (Portable Document Format):** The universally accepted file format for the research papers inputted into the system.
- **UI (User Interface):** The interactive frontend dashboard built using the Streamlit Python framework.
- **Vector Database:** A specialized database (specifically ChromaDB in this system) explicitly engineered to store, manage, and rapidly search high-dimensional numerical representations of text data.
- **Embeddings:** The complex mathematical process of converting plain English text into continuous numerical vectors (using HuggingFace's `all-MiniLM-L6-v2` model) so that the AI can computationally understand semantic meaning and contextual relationships.

2. Overall Description

2.1 Product Perspective

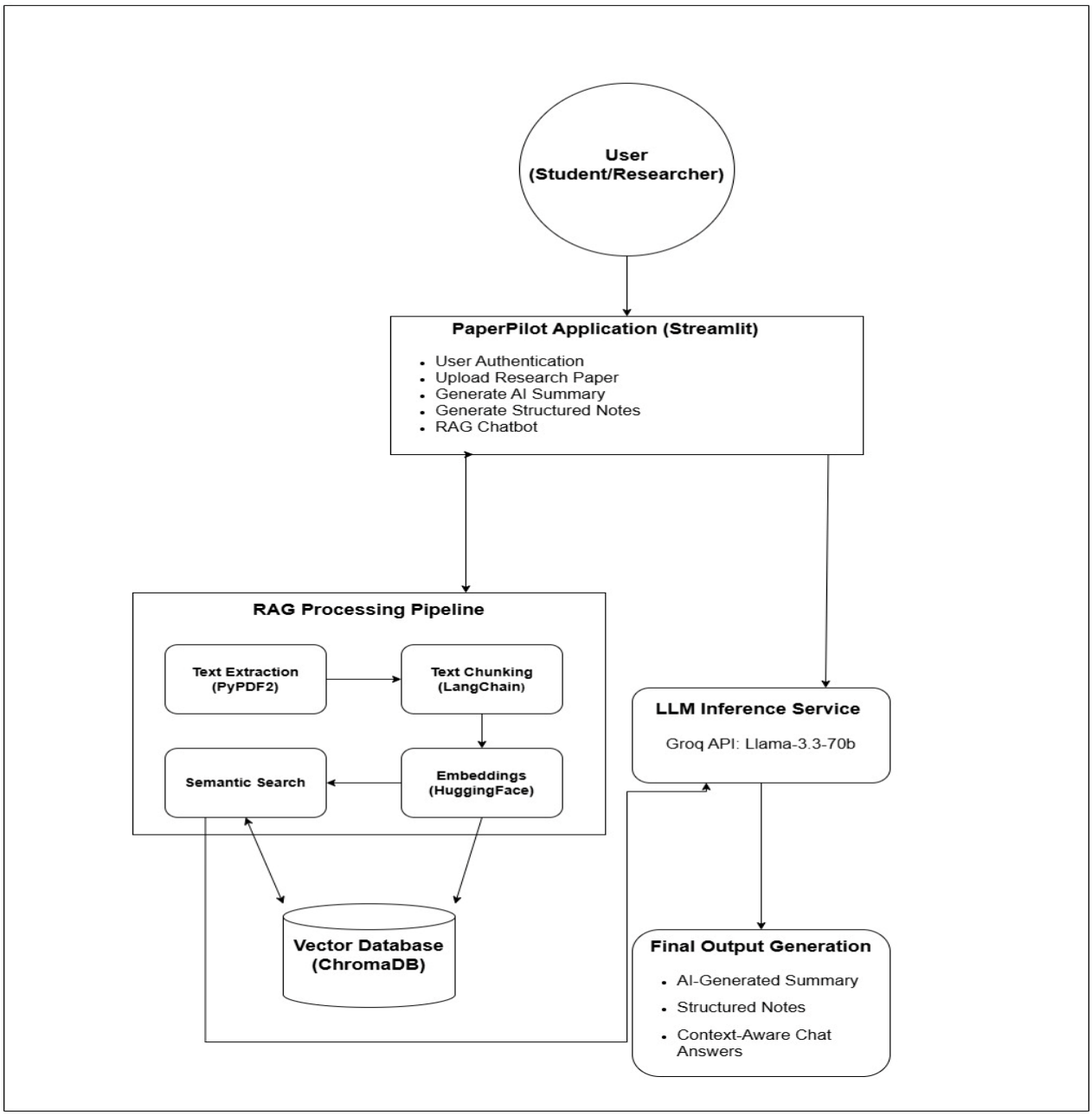
PaperPilot operates as a standalone, modular web application engineered for high-performance text analysis. While the frontend and vector storage run locally (or on a hosted server), the heavy computational language processing is outsourced to cloud-based inference APIs to ensure rapid response times without requiring high-end local GPU hardware.

The system integrates several state-of-the-art frameworks to form a cohesive pipeline:

- **Frontend/UI Layer:** Developed exclusively using **Streamlit** (Python), providing a reactive, component-driven user interface that updates dynamically without page reloads.

- **Backend Orchestration & Pipeline:** Powered by the **LangChain** framework, which acts as the central nervous system bridging the document loader, text splitters, and the LLM.
- **AI Integration (Inference Engine):** Utilizes the **Groq API** running the highly advanced llama-3.3-70b-versatile open-source model. Groq's LPU (Language Processing Unit) architecture guarantees near-instantaneous token generation.
- **Vector Database: ChromaDB** is deployed locally to store high-dimensional mathematical representations of text, allowing for lightning-fast semantic similarity searches.
- **Embeddings:** The **HuggingFace** all-MiniLM-L6-v2 model is utilized to map textual data into dense vector spaces, capturing the semantic meaning of the uploaded documents.

PAPER PILOT SYSTEM ARCHITECTURE (RAG PIPELINE)



2.2 Product Functions

The system autonomously executes the following core functionalities, transitioning from raw data ingestion to intelligent output generation:

1. **Stateful User Authentication & Session Management:** A secure login and registration portal that restricts unauthorized access. The system utilizes st.session_state to preserve user workspaces across interface interactions.
2. **Robust Document Ingestion:** A secure upload module that strictly filters for PDF formats, temporarily storing the document in the server's local directory for processing.
3. **Automated Text Chunking & Vectorization:** The system extracts raw strings using PyPDF2 and strategically splits the text into 1000-character overlapping chunks to maintain semantic integrity before saving them into ChromaDB.
4. **Context-Bound Summarization:** Generates an immediate, high-level abstract of the paper by feeding the initial 5000 characters of the document context directly into the Groq LLM.
5. **Structured Knowledge Extraction (Notes):** Applies specific prompt engineering to force the LLM to output segmented, highly readable study notes categorized distinctly into Introduction, Key Points, and Conclusion.
6. **Interactive RAG-Powered Chatbot:** A conversational interface that maintains message history and queries the vector database (k=3 similarity search) to answer highly specific user questions based *only* on the uploaded document.

2.3 User Classes

The system caters primarily to the following user classes:

- **Primary End-Users (Researchers, Students, Academics):** Individuals looking to dramatically reduce the time spent reading extensive academic literature. They require an intuitive, lag-free interface to upload papers and extract insights without needing technical knowledge of the underlying AI pipeline.
- **System Administrators (Optional/Future Scope):** Technical personnel responsible for monitoring API usage, maintaining the Groq API keys within the environment variables, and ensuring the local ChromaDB instances are periodically flushed to optimize storage.

2.4 Operating Environment

PaperPilot is highly portable and OS-agnostic due to its Python-based architecture.

- **Operating System:** Fully compatible with Windows, Linux, and macOS environments.
- **Browser Compatibility:** Optimized for modern web browsers (Google Chrome, Microsoft Edge, Mozilla Firefox, Safari) running modern V8 JavaScript engines.
- **Deployment Platform:** Currently executes on a local server environment (localhost:8501) via the Streamlit runtime, with full compatibility for future deployment on Streamlit Community Cloud or AWS/GCP instances.

2.5 Constraints

The development and operation of PaperPilot are bound by the following technical constraints:

- **API Dependency & Internet Requirement:** The system relies entirely on the Groq API for LLM inference. A stable internet connection is strictly required; offline operation is limited to UI rendering.
- **File Format Restriction:** The PyPDF2 extraction module strictly supports standard .pdf files. It cannot process Word documents (.docx) or plain text files (.txt) in its current iteration.
- **No OCR Capabilities (Scanned PDFs):** The system extracts embedded text. It cannot read text from scanned images within a PDF (Optical Character Recognition is not integrated).
- **Storage and Size Limitations:** To prevent memory overflow and excessive embedding computation, file uploads are capped at a maximum of 200MB.

2.6 Assumptions

The successful execution of the project assumes the following:

- **API Configuration:** It is assumed that a valid Groq API key is correctly configured and securely stored in the root directory's .env file prior to execution.
- **Service Uptime:** The system assumes consistent uptime and availability from external dependencies, specifically HuggingFace (for embedding models) and the Groq Cloud platform.
- **User Competency:** It is assumed that users possess fundamental digital literacy, capable of downloading PDFs from academic journals and utilizing a standard web upload interface.

3. System Features

3.1 User Authentication

Description: A robust state-management system that allows users to securely log in or create a new account, ensuring that individual workspaces remain isolated.
Functional Requirements:

- **Credential Input:** The interface provides secure text input fields for email and masked passwords.
- **Stateful Validation:** The system dynamically validates user credentials against a backend dictionary stored in memory.
- **Error Handling:** Displays visually distinct error banners (via Streamlit's st.error) upon detecting invalid login attempts or unregistered emails.
- **Account Registration:** Facilitates the creation of new accounts, immediately updating the active st.session_state.users registry without requiring a hard server reboot.

3.2 Upload Research Paper

Description: A dedicated document parsing module where users can upload complex PDF research papers for backend vectorization.
Functional Requirements:

- **Interactive Upload Interface:** Features a drag-and-drop or file-browser uploader component.
- **Strict MIME Validation:** System programmatically enforces a strict .pdf file extension policy, immediately rejecting unsupported formats (e.g., .docx, .txt).
- **Asynchronous Processing Feedback:** Displays an interactive loading spinner while the PyPDF2 library extracts raw text and LangChain chunks the data into the vector database.
- **Ephemeral Storage:** The file is securely stored in a temporary local directory (temp_pdf) solely for the duration of the processing phase, optimizing memory usage.

3.3 Generate AI Summary

Description: An algorithmic summarization engine that autonomously generates a concise, readable abstract of the uploaded academic paper.
Functional Requirements:

- **Action Trigger:** A dedicated action button that initiates the LLM inference sequence.
- **Contextual Parsing:** The system extracts the first 5,000 characters of the document to serve as the prompt context, circumventing token-limit bottlenecks while capturing the core abstract.
- **Dynamic Output Generation:** AI processes the extracted text and streams the generated summary directly onto the primary dashboard interface.

3.4 Generate Structured Notes

Description: An intelligent parsing feature that forces the LLM to extract and categorize critical academic data into a rigid, readable structure.

Functional Requirements:

- **Action Trigger:** A specialized button that sends a highly engineered prompt to the Groq API.
- **Structured Output:** The system mandates the Large Language Model to categorize the output strictly into predefined subheadings:
    - **Introduction:** The core thesis and background of the paper.
    - **Key Points:** Bulleted extraction of methodologies, data points, or critical arguments.
    - **Conclusion:** The final outcomes and future scope discussed in the research.

3.5 RAG Chatbot

Description: The core retrieval-augmented module allowing users to ask hyper-specific questions, ensuring answers are derived entirely from the uploaded document rather than pre-trained general knowledge.

Functional Requirements:

- **Query Input:** A persistent chat input field dedicated to user queries.
- **Semantic Similarity Search:** Upon querying, the system cross-references the input against the local ChromaDB vector store, retrieving the top 3 (k=3) most semantically relevant text chunks.
- **Grounded Answer Generation:** The retrieved context and the user's query are merged into a synthesized prompt and sent to the LLM, returning a highly accurate, hallucination-free response.
- **Conversational Memory:** The system utilizes st.session_state.messages to retain and display the ongoing chat history for a seamless user experience.

4. External Interface Requirements

4.1 User Interface

- **Minimalist Dashboard Design:** A clean, distraction-free central dashboard built entirely via Streamlit, optimizing readability for dense academic texts.
- **Stateful Sidebar Navigation:** A persistent left-hand navigation menu utilizing radio buttons to dynamically switch between system states:
    - Upload Paper
    - Generate Summary
    - Generate Notes
    - Chatbot

4.2 Hardware Interface

- **Client-Side:** Requires only a standard PC, laptop, or tablet. Because the heavy computational loads (Vectorization and LLM Inference) are handled server-side and via cloud APIs, the client hardware requires minimal RAM and processing power.
- **Server-Side (Host):** Requires sufficient local RAM to host the Python runtime, Streamlit server, and the in-memory ChromaDB vector collections during active sessions.

4.3 Software Interface

- **Core Environment:** Python 3.9+ runtime environment.
- **Frontend Framework:** Streamlit library for rendering reactive web components without HTML/CSS.
- **AI & Machine Learning APIs:** * Groq API for ultra-low latency LLM inference.
    - HuggingFace API for accessing the all-MiniLM-L6-v2 embedding models.

5. Non-Functional Requirements

5.1 Performance

- **Inference Latency:** Leveraging Groq's Language Processing Units (LPUs), the system is mandated to return AI-generated responses (summaries or chat answers) in under 3 seconds.
- **Processing Efficiency:** The text chunking strategy (1000 characters with 200 overlap) ensures that document vectorization happens rapidly without causing memory overflow or browser timeouts.

5.2 Security

- **Environment Variable Isolation:** Critical cryptographic data, specifically the Groq API Key, is strictly stored in a `.env` file and completely excluded from version control (e.g., `.gitignore`) to prevent security breaches.
- **Authentication Integrity:** Only authenticated users can access the dashboard. Unauthorized users attempting to bypass the login state are forcefully redirected.

5.3 Usability

- **Intuitive Workflow:** Designed with a zero-learning-curve philosophy. Users simply upload a document and click buttons to generate insights.
- **Proactive Error Prevention:** The system actively disables or warns users if they attempt to use the Summary, Notes, or Chatbot features *before* successfully uploading a PDF.

5.4 Reliability

- **Stable Response Generation:** Prompt engineering ensures the LLM does not hallucinate data if the answer is not found within the PDF text.
- **Robust Error Handling:** The system includes `try-except` blocks (or functional equivalents) to catch missing API keys, unreadable PDFs, or network timeouts, returning human-readable error messages (`st.warning` or `st.error`) instead of crashing.

5.5 Scalability

- **Multi-User Extension:** The session-state architecture is inherently designed to isolate user data, allowing multiple users to access the platform concurrently without data cross-contamination.
- **Cloud Deployment Readiness:** The application is architected as a stateless frontend (excluding temp files), making it fully prepared for containerized deployment on platforms like Streamlit Community Cloud, AWS, or Heroku.

6. Future Enhancements

- **Multi-File Upload & Cross-Referencing:** Enabling researchers to upload multiple PDFs simultaneously and ask the RAG chatbot to compare methodologies or results across different papers.
- **Export & Download Functionality:** Implementing a module to allow users to download the AI-generated summaries and structured notes directly as a formatted `.pdf` or `.docx` file for offline use.
- **Cloud Storage Integration:** Transitioning from temporary local storage to robust cloud buckets (e.g., AWS S3) for persistent document hosting.
- **User History & Analytics Tracking:** Integrating a backend database (such as PostgreSQL or MongoDB) to permanently save user chat histories, previously uploaded vectors, and usage analytics across multiple sessions.